Loss-Conditional PINNs: One Training Run for a Parametric PDE Family

Anonymous Author(s)
Affiliation
Address
email

Abstract

Physics-informed neural networks (PINNs) require a fresh training run for any change in the residual: a new wavenumber, a new mobility ratio, a new loss-weight balance. Operator-learning methods (FNO, DeepONet) amortise across a parametric family but require solver-generated paired (input, solution) examples *in their training loop*. We close that gap. LC-PINN is a single residual-loss training run that produces a continuous family of PDE solutions over a parameter λ — a physical scalar coefficient or a loss-weight vector — without paired training data, test-time adaptation, or per-λ retuning. Reference solutions are used only to compute test-time error. The construction adapts loss-conditional networks (Dosovitskiy and Djolonga, 2020) to PINN training: λ becomes a network input drawn fresh from a fixed prior at every optimisation step. We prove a λ-invariance result — at a global optimum of the λ-expected loss the network is a per-λ residual minimiser for p_λ-almost every λ — with a finite-capacity corollary that bounds the suboptimality at every λ in the support. Across 1D, 2D, and 3D parametric Helmholtz, 1D stationary Schrödinger with a multiplicative parametric potential, and loss-weight amortisation on viscous Burgers and Buckley–Leverett, LC reaches rel-L^2 within a factor of two of the strongest single-λ baseline. On 1D Helmholtz it is the only method whose rel-L^2 stays within one order of magnitude across $k \in [1, 10]$ — per-k retrained baselines vary by three.

1 Introduction

Physics-Informed Neural Networks (PINNs; Raissi et al., 2019) fit a single neural network to a single PDE by backpropagating the residual of the PDE itself. Two training-time choices control the result and are notoriously difficult to set: the relative weights of the loss terms (residual / boundary / initial / data), and the value of any physical parameter that enters the operator. Both are typically *tuned*: weights via grid search or self-adaptive scheduling (McClenny and Braga-Neto, 2020; Bischof and Kraus, 2021; Wang et al., 2024), parameters via outright retraining at every value the practitioner cares about. Either way, every change in the residual costs a fresh training run.

This paper removes that cost. We show that a PINN can be trained *once* so that the same network covers an entire family of loss-weight balances or an entire range of a physical coefficient, with no test-time adaptation, no per-λ retuning, and no solver-generated paired training data — one residual-loss training run, then a single forward pass per query. Reference solutions appear only at evaluation, to compute rel-L^2 error.

A separate stream of work in scientific machine learning attacks the same problem from the operator-learning angle: methods like FNO (Li et al., 2021) and DeepONet (Lu et al., 2021) amortise across a family of PDE instances by treating the right-hand side or the initial condition as a function-valued input. They produce a single network that responds to a continuum of inputs — but they require a

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

training distribution of *paired* (input, solution) examples generated by an external solver; the PDE residual itself plays no role in training.

This paper sits between those two worlds. We adopt the PINN residual-loss machinery and turn the parameter into a network input, so a single PINN-style training run produces a continuum of solutions indexed by λ. The construction generalises the loss-conditional networks of Dosovitskiy and Djolonga (2020) — originally proposed for image-generation loss-mixture amortisation — to the parametric-PDE setting. Concretely:

> *LC-PINN is a residual-loss training run that produces a continuous family of PDE solutions over a parameter λ — a physical scalar coefficient or a loss-weight vector — with no solver- generated paired training data, no test-time adaptation, and no per-λ retuning.*

The same architecture serves two distinct uses, the parametric-coefficient mode (λ a physical scalar) and the loss-weight mode (λ a weight vector); both are reported as separate experiment families below.

Contributions.

- We formulate the LC-PINN (Equations (2) and (3)) and prove a λ-invariance result: at a global optimum of the λ-expected loss, the conditional network is p_λ-a.e. a residual-minimiser of the parametric PDE indexed by λ (Proposition 1). A finite-capacity corollary bounds the per-λ suboptimality by the realisable residual budget plus the optimisation gap (Corollary 2).
- On 1D parametric Helmholtz ($k \in [1, 10]$) the amortised LC-PINN reaches rel-$L^2 \sim 9{\times}10^{-4}$ on the grid mean and stays within one order of magnitude of its own best across the entire parameter range. Per-k retrained SA-PINN/ReLoBRaLo baselines vary by three orders of magnitude across k — occasionally beating LC by a factor of ~ 200 at one k, then catastrophically failing at another. LC is the only method whose rel-L^2 stays within one order of magnitude across the entire k-range (Section 5.1).
- On 2D parametric Helmholtz on the unit square and 3D parametric Helmholtz on the unit cube the same construction extends without architectural changes: rel-$L^2 \sim 2.4 \times 10^{-2}$ in 2D ($\sim 3\times$ better than per-k SA-PINN baselines, Section 5.3) and $\sim 1.7{\times}10^{-2}$ in 3D ($\sim 4\times$ better than per-k SA-PINN, Section 5.4).
- On a 1D stationary Schrödinger problem with a parametric harmonic potential ($\alpha \in [0.5, 10]$), where the parameter enters multiplicatively against u rather than as a constant zeroth-order coefficient, the same FiLM+L-BFGS construction transfers without architectural changes (Section 5.2); this rules out the suspicion that the lift is specific to the wave operator.
- On viscous Burgers and capillarity-corrected Buckley–Leverett, LC-PINN is within a factor of two of the strongest single-shot baseline while parameterising the entire loss-weight family from one trained network, breaking even on training compute after ~ 4 retrainings and amortising $\sim 29\times$ at $K = 100$ (Section 5.5).
- Ablations on the K-shot inference budget, per-step λ-sample count, and uniform vs. log-uniform sampling distributions (Section 5.6).

2 Related work

Adaptive loss weighting in PINNs. The fragility of the composite-loss objective in the original PINN formulation (Raissi et al., 2019) has motivated a line of methods that adapt the residual / boundary / data weights during training. Self-adaptive PINN (SA-PINN) of McClenny and Braga-Neto (2020) attaches a trainable per-collocation-point weight that is updated by ascent on the residual, yielding an effective focusing of the network on hard-to-fit regions; their "points-with-weights" formulation is essentially a soft hard-example-mining for the residual. ReLoBRaLo (Bischof and Kraus, 2021) re-balances loss-term weights from running ratios of the absolute loss values, smoothed by an exponential moving average. Causal-PINN (Wang et al., 2024) weights the residual by a time-causal mask so the network does not collapse onto late-time dynamics before fitting the initial-time

region. These all produce *one* solution for *one* (adaptively-found) weighting and would need to be retrained at every new weighting the practitioner wishes to inspect.

Operator learning. A parallel line of work has built neural operators that map a function-valued input (boundary condition, forcing term, initial condition) to a function-valued solution. The Fourier Neural Operator (FNO) (Li et al., 2021) parameterises the operator in spectral space: linear layers on the lowest Fourier modes, plus a residual 1×1 convolution. DeepONet (Lu et al., 2021) factorises the operator into a branch network (encoding the input function at sensor points) and a trunk network (encoding the spatial coordinate), recombined by an inner product. Both methods amortise across a family of problems but require a precomputed dataset of (input, solution) pairs from a classical solver—they are supervised learners in function space. The PINN residual itself plays no role in their training. PI-DeepONet (Wang et al., 2021) re-introduces the residual but stays inside the operator-learning framing, treating the input function as the parameter rather than a scalar coefficient. Our LC-PINN sits between these two camps: it amortises like operator learning but uses the residual loss like a PINN.

Loss-conditional networks. Dosovitskiy and Djolonga (2020) introduced the loss-conditional networks idea in image generation: a single generator, conditioned on the loss weights of a multi-objective composite reconstruction loss, produces images that interpolate the tradeoff surface (sharpness vs. perceptual fidelity vs. adversarial realism). The network is trained by sampling the weight vector from a fixed prior at every step and applying the freshly-sampled weights to the per-instance reconstruction loss. LC-PINN takes the same conditional-on-weights idea and applies it to the PDE-residual setting; the formal λ-invariance result (Section 3.3) is, to our knowledge, the first analysis of the optimum-set structure of this construction in the residual-loss case.

Hypernetworks and meta-learning for PINNs. A separate amortisation strategy uses hypernetworks (Ha et al., 2017) or MAML-style meta-learning (Finn et al., 2017) to produce per-instance PINN weights. PI-MAML (Liu et al., 2022) demonstrates the meta-learning angle on parametric PDEs. These methods amortise at the level of network weights rather than a network input. They typically require a training distribution of full PDE *tasks* and a per-task inner-loop adaptation step at test time; LC-PINN trains a single network with no inner loop and produces predictions for every λ in one forward pass.

Extrapolation in λ vs. in physical parameters. In the loss-weight mode the network's role is closest to Dosovitskiy and Djolonga (2020); in the parametric-coefficient mode it is closest to FNO / DeepONet but parameterised over a low-dimensional scalar input rather than a function-valued one. The two modes share an architecture; the choice of λ is task-driven.

3 Method

3.1 Loss-conditional formulation

A standard PINN (Raissi et al., 2019) approximates the solution of a single PDE $Fu = 0$ with auxiliary boundary, initial, and (optionally) data conditions $\{Bu = g_B,\, Iu = g_I,\, Du = g_D\}$ by minimising the weighted composite loss

$$\mathcal{L}^{\mathrm{PINN}}(\theta;\, w) = w_F \left\|Fu_\theta\right\|_{L^2}^2 + w_B \left\|Bu_\theta - g_B\right\|_{L^2}^2 + w_I \left\|Iu_\theta - g_I\right\|_{L^2}^2 + w_D \left\|Du_\theta - g_D\right\|_{L^2}^2, \quad (1)$$

in the network parameters θ. The choice $w = (w_F, w_B, w_I, w_D)$ controls a strong tradeoff between satisfying the residual and matching the auxiliary constraints; in practice the weights are tuned by grid search or by an adaptive schedule (SA-PINN (McClenny and Braga-Neto, 2020), ReLoBRaLo (Bischof and Kraus, 2021), Causal-PINN (Wang et al., 2024)).

We replace the fixed-w problem with a *family* of problems indexed by a parameter vector $\lambda \in \Lambda$, where $\Lambda \subseteq \mathbb{R}^{d_\lambda}$ is either (i) the set of admissible loss weights itself ($\lambda = w$, $d_\lambda = 4$ for Burgers and BL), or (ii) a physical parameter of the PDE such as a wavenumber or mobility ratio ($\lambda = k$, $d_\lambda = 1$ for Helmholtz). The *loss-conditional* PINN (LC-PINN) is a single network

$$u_\theta : \Omega \times \Lambda \to \mathbb{R}, \qquad (x, \lambda) \mapsto u_\theta(x, \lambda), \quad (2)$$

trained to minimise the λ-expectation of the per-instance loss,

$$J(\theta) = \mathbb{E}_{\lambda \sim p_\lambda}\left[\mathcal{L}\big(\lambda;\, u_\theta(\cdot, \lambda)\big)\right], \tag{3}$$

where p_λ is a fixed sampling distribution (uniform on Λ unless stated otherwise). The architectural cost is one extra input of width d_λ at the first layer; the per-step compute is one forward/backward pass at a freshly drawn λ. This generalises the loss-conditional networks of Dosovitskiy and Djolonga (2020) from generative loss-mixture amortisation to the residual-loss setting in which the "loss mixture" is the PDE residual itself.

3.2 Inference: K-shot averaging across λ

At evaluation time we report a single rel-L^2 error per task. For the loss-weight conditioning case there is no canonical "true" weight, so following Dosovitskiy and Djolonga (2020) we average network predictions over K samples of λ drawn from p_λ:

$$\hat{u}(x) = \frac{1}{K}\sum_{i=1}^{K} u_\theta(x, \lambda^{(i)}), \qquad \lambda^{(i)} \overset{\text{i.i.d.}}{\sim} p_\lambda. \tag{4}$$

This K-shot averaging exposes the variance structure of the LC-PINN across the loss-weight family and underpins the amortisation analysis in Section 5: a baseline PINN must be retrained K times to produce K predictions; the LC-PINN produces all K from a single training run.

3.3 λ-invariance at the LC optimum

The empirical claim of this paper is that a single LC-PINN $u_\theta(x, \lambda)$, trained with λ sampled from a probability measure p_λ, parameterises the parametric PDE family $\{F_\lambda u = 0\}_{\lambda \in \Lambda}$ *simultaneously* in λ. The following proposition makes precise the sense in which this holds at a global optimum of the LC-PINN objective.

Setup. Let $\Lambda \subseteq \mathbb{R}^{d_\lambda}$ be the parameter set, p_λ a probability measure on Λ with full support, and $\Omega \subseteq \mathbb{R}^{d_x}$ the spatial–temporal domain with sampling measure p_Ω. Each $\lambda \in \Lambda$ indexes a residual operator F_λ together with associated boundary, initial, and (optional) data-fit operators $\{B_\lambda, I_\lambda, D_\lambda\}$. For a candidate field $v : \Omega \to \mathbb{R}$, write the per-$\lambda$ functional

$$\mathcal{L}(\lambda; v) \;=\; w_F \left\|F_\lambda v\right\|^2_{L^2(\Omega, p_\Omega)} + w_B \left\|B_\lambda v\right\|^2_{L^2(\partial\Omega)} + w_I \left\|I_\lambda v\right\|^2_{L^2(t=0)} + w_D \left\|D_\lambda v\right\|^2_{L^2},$$

with strictly positive loss weights (w_F, w_B, w_I, w_D). The LC-PINN training objective is the p_λ-expectation

$$J(\theta) \;=\; \int_\Lambda \mathcal{L}\big(\lambda;\, u_\theta(\,\cdot\,, \lambda)\big)\, dp_\lambda(\lambda).$$

Assume:

- (A1) the network class $\{u_\theta(\cdot, \lambda) : \theta \in \Theta\}$ contains, for every λ, a residual-minimiser $u^\star_\lambda \in \arg\min_v \mathcal{L}(\lambda; v)$ with $\mathcal{L}(\lambda; u^\star_\lambda) = 0$;
- (A2) the map $\lambda \mapsto \mathcal{L}(\lambda; u_\theta(\cdot, \lambda))$ is dp_λ-integrable for every θ;
- (A3) u_θ is jointly continuous in (x, λ).

Proposition 1 (Residual-minimiser at the LC optimum)**.** *Let $\theta^\star \in \arg\min_\theta J(\theta)$. Under* (A1)–(A3)*, for p_λ-almost every $\lambda \in \Lambda$ the section $u_{\theta^\star}(\cdot, \lambda)$ is a residual-minimiser of the parametric PDE indexed by λ:*

$$\mathcal{L}\big(\lambda;\, u_{\theta^\star}(\,\cdot\,, \lambda)\big) \;=\; 0 \qquad \textit{for } p_\lambda\textit{-a.e. } \lambda \in \Lambda.$$

In particular, on $\operatorname{supp}(p_\lambda)$ the LC-PINN section satisfies $F_\lambda u_{\theta^\star}(\cdot, \lambda) = 0$ in $L^2(\Omega, p_\Omega)$ and the auxiliary constraints to the same precision.

Sketch proof. The integrand $\lambda \mapsto \mathcal{L}(\lambda; u_\theta(\cdot, \lambda))$ is non-negative for every θ and every λ, since each summand is a squared L^2-norm. By (A1) the per-λ minimum is zero, so $\inf_\theta J(\theta) \geq 0$ and the lower bound is attainable: any measurable selector $\lambda \mapsto \theta_\lambda$ with $u_{\theta_\lambda}(\cdot, \lambda) = u^\star_\lambda$ exists pointwise

under (A1), and a single $\theta^\star$ achieving $J(\theta^\star) = 0$ exists when the network class is rich enough to parameterise the family $\{u^\star_\lambda\}_{\lambda\in\Lambda}$ jointly (e.g. universal approximation in $C(\Omega \times \Lambda)$). For such $\theta^\star$,

$$0 \;=\; J(\theta^\star) \;=\; \int_\Lambda \mathcal{L}\big(\lambda;\, u_{\theta^\star}(\,\cdot\,,\lambda)\big)\, dp_\lambda(\lambda),$$

and the integrand is non-negative; the standard "vanishing-integral of a non-negative function" lemma then gives $\mathcal{L}(\lambda; u_{\theta^\star}(\cdot,\lambda)) = 0$ for p_λ-a.e. λ. □

Remark 1 (where the support assumption matters). The conclusion is a p_λ-a.e. statement on Λ: nothing prevents the LC-PINN from being arbitrarily wrong on a p_λ-null set, including the boundary of Λ if p_λ has no mass there. In our experiments $p_\lambda = \mathrm{Uniform}(\Lambda)$ has full support, so the p_λ-a.e. statement coincides with Lebesgue-a.e. on Λ.

Corollary 2 (Finite-capacity LC bound)**.** *Replace assumption* (A1) *by the weaker* ε-residual capacity *condition: there exists* $\bar\theta \in \Theta$ *with* $\mathcal{L}(\lambda; u_{\bar\theta}(\cdot,\lambda)) \le \varepsilon$ *for* p_λ*-a.e.* λ*. Suppose the optimiser reaches an objective value within* δ *of the infimum:* $J(\theta^\star) \le \inf_\theta J(\theta) + \delta$*. Then for every* $\eta > 0$*,*

$$\Pr_{\lambda\sim p_\lambda}\Big[\mathcal{L}\big(\lambda;\, u_{\theta^\star}(\,\cdot\,,\lambda)\big) \;>\; \varepsilon + \eta\Big] \;\le\; \frac{\delta}{\eta},$$

i.e. on a p_λ*-set of mass* $\ge 1 - \delta/\eta$ *the per-*λ *residual loss is at most* $\varepsilon + \eta$*.*

Sketch. By definition of $\bar\theta$ and (A2), $\inf_\theta J(\theta) \le J(\bar\theta) \le \varepsilon$, so $J(\theta^\star) \le \varepsilon + \delta$. Let $g(\lambda) = \mathcal{L}(\lambda; u_{\theta^\star}(\cdot,\lambda)) - \varepsilon$, which is p_λ-a.e. bounded below by $-\varepsilon$ but in particular below by zero on the p_λ-set where the comparator $\bar\theta$ achieves the ε-bound. Markov's inequality applied to the non-negative part $g_+ = \max(g, 0)$ gives $\Pr[g_+ > \eta] \le \mathbb{E}[g_+]/\eta \le \delta/\eta$. □

This is the practically observable form of the λ-invariance result: it bounds the per-λ loss for *any* suboptimal $\theta^\star$ by the realisable residual budget plus the optimisation gap, and matches the per-λ rel-L^2 tables in Section 5.

Remark 2 (sampling-distribution invariance). If p_λ and $\tilde p_\lambda$ are two probability measures with the same support $\Lambda_0 \subseteq \Lambda$, the sets of *a.e.-residual-minimisers* on Λ_0 coincide. So the optimum-set of the LC objective on $\mathrm{supp}(p_\lambda)$ does not depend on the precise sampling law—only on the support. This is an asymptotic statement; at finite training budget the law of p_λ governs which slices receive enough gradient signal, which we observe empirically (Section 5.6).

Remark 3 (sample complexity per step). At each Adam step the gradient is built from n_λ i.i.d. draws of $\lambda \sim p_\lambda$. Conditional on θ, the per-step gradient estimator $\hat g_{n_\lambda} = \frac{1}{n_\lambda}\sum_{i=1}^{n_\lambda} \nabla_\theta \mathcal{L}(\lambda_i; u_\theta(\cdot,\lambda_i))$ is unbiased for $\nabla_\theta J(\theta)$, and if the per-λ gradient norm is p_λ-essentially bounded by G then $\Pr[\|\hat g_{n_\lambda} - \nabla J\|_\infty > t] \le 2\exp(-n_\lambda t^2/2G^2)$ by Hoeffding. The variance of the estimate is $O(1/n_\lambda)$, so the regime $n_\lambda \sim 4$ empirically chosen in Section 5.6 corresponds to the sweet spot between gradient noise (small n_λ) and budget concentration (large n_λ at fixed total step count).

3.4 Network and training details

All experiments use a fully-connected $\tanh$ network with hidden widths $[64, 64, 64, 64]$, Xavier initialisation, and a single real-valued output head. Two conditioning routes are used: in the *loss-weight* mode (Burgers, BL) λ is concatenated to the spatial–temporal coordinates at the input layer following Dosovitskiy and Djolonga (2020); in the *parametric-coefficient* mode (Helmholtz 1D/2D/3D, Schrödinger) λ is mapped to FiLM (Perez et al., 2018) scale/shift parameters that modulate every hidden layer, followed by an L-BFGS finishing pass with a revert-on-worse safety check. The FiLM+L-BFGS combination is what buys the order-of-magnitude lift over plain concat in the parametric-coefficient mode (Appendix C documents both directions). Training uses Adam ($\eta = 10^{-3}$) with cosine annealing and gradient-norm clipping at 1.0; each step samples $\lambda \sim p_\lambda$ and evaluates the per-instance loss at the current batch of collocation points. λ is normalised to $[-1, 1]$ before entering the network. Implementation: PyTorch 2.9 on Apple M4 Max via the MPS backend. Full hyperparameters and per-equation collocation budgets are listed in Appendix A.

4 Experiments

We evaluate LC-PINN on four parametric PDE families spanning the two amortisation modes of Section 3: a loss-weight family on viscous Burgers and Buckley–Leverett ($\lambda = w$, $d_\lambda = 4$); a parametric-coefficient family on Helmholtz in 1D, 2D, and 3D ($\lambda = k$, the wavenumber); and 1D Schrödinger with parametric harmonic potential ($\lambda = \alpha$, trap stiffness). Baselines: SA-PINN (McClenny and Braga-Neto, 2020), ReLoBRaLo (Bischof and Kraus, 2021), Causal-PINN (Wang et al., 2024) (Burgers/BL only), FNO (Li et al., 2021), and PI-DeepONet (Wang et al., 2021). Full equations, manufactured references, training budgets, and baseline hyperparameters are in Appendix A; collocation counts are listed in Table 8.

Evaluation protocol. Loss-weight mode (Burgers, BL): K-shot averaged rel-L^2 at $K = 100$ unless noted; the K-eval ablation in Section 5.6 sweeps $K \in \{25, 50, 100, 200, 400\}$. Parametric-coefficient mode (Helmholtz): rel-L^2 at a fixed k-grid (1D: five points $\{1.00, 3.25, 5.50, 7.75, 10.00\}$; 2D: three points $\{1.00, 5.50, 10.00\}$; 3D: three points $\{1, 3, 5\}$), averaged across the grid and then over seeds. Schrödinger uses a 20-point α-grid for LC, restricted to the SA-PINN training points $\{0.5, 5.0, 10.0\}$ when reporting alongside SA. All numbers are mean $\pm$ std over 4 random seeds. Wall times are measured on a single Apple M4 Max via the PyTorch MPS backend.

5 Results

We organise the results around the two amortisation modes, leading with the parametric-coefficient mode where the comparison against operator-learning is sharpest. The parametric-coefficient mode (Helmholtz, 1D and 2D) is reported as the rel-L^2 averaged across a fixed five-point k-grid; the loss-weight mode (Burgers, BL) is reported with K-shot averaged rel-L^2. All numbers are mean $\pm$ std over 4 random seeds (2 seeds for the per-k baselines in 2D, where each baseline retrains from scratch at every k). Figure 1 previews the headline tradeoff.

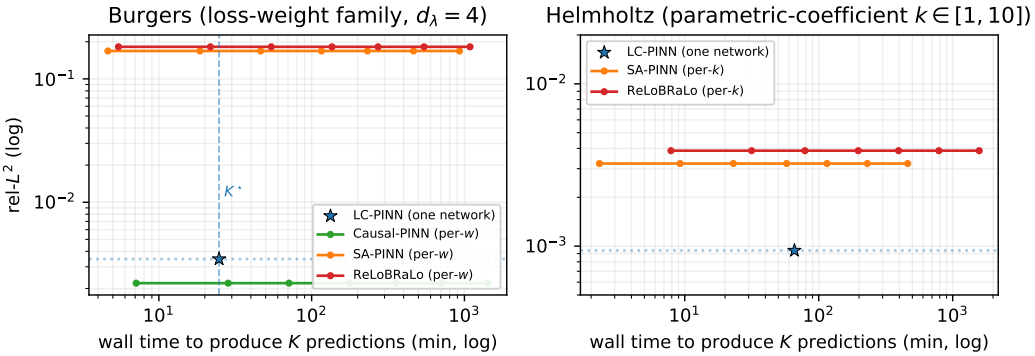

Figure 1: Wall-time / accuracy frontier across K (the number of λ values evaluated). LC-PINN (blue $\star$) trains once and serves any K for free; per-λ baselines walk to the right by a factor K. On Helmholtz LC pareto-dominates from $K = 1$ onwards; on Burgers Causal-PINN edges LC per-point ($\sim 1.6\times$) but crosses LC's wall time at $K^\star \approx 4$ (vertical dashed line: any baseline marker right of it costs more for K predictions than LC).

5.1 1D Helmholtz, parametric-coefficient mode

Table 1 reports rel-L^2 averaged across the five-point k-grid. LC-PINN's input is the wavenumber k itself (normalised to $[-1, 1]$); the baselines are retrained at each k_{train}.

A single LC-PINN training run, using FiLM conditioning (Perez et al., 2018) on k and an L-BFGS finishing pass, produces a network that covers all $k \in [1, 10]$ at rel-$L^2 \sim 9 \times 10^{-4}$. The grid-mean comparison favours LC by a factor of ~ 3.4 over SA-PINN and ~ 4.1 over ReLoBRaLo, and at lower total wall time than a single run of all five per-k retrainings combined.

Table 1: 1D Helmholtz, rel-L^2 averaged across $k \in \{1.00, 3.25, 5.50, 7.75, 10.00\}$. LC-PINN is one trained network evaluated at each k; baselines are five separate retrainings, one per k.

Method	rel-L^2 (mean $\pm$ std)	wall time
LC-PINN (FiLM+L-BFGS, one network)	$\mathbf{9.39 \times 10^{-4} \pm 1.18 \times 10^{-4}}$	65.9 min/seed
PI-DeepONet (one network)	$2.21 \times 10^{-2} \pm 3.18 \times 10^{-2}$	61.8 min/seed
SA-PINN (per-k, 5 retrainings)	$3.23 \times 10^{-3} \pm 2.90 \times 10^{-3}$	11.4 min/k
ReLoBRaLo (per-k, 5 retrainings)	$3.87 \times 10^{-3} \pm 2.82 \times 10^{-3}$	7.9 min/k

Per-k breakdown: who wins where. The grid mean hides a more nuanced picture. Table 2 splits the rel-L^2 across the five evaluation k values for each method. Per-k retrained baselines are remarkable at the easy end of the family — ReLoBRaLo trained at $k=1$ achieves 5.7×10^{-6}, beating LC by more than two orders of magnitude — but they collapse at the difficult end: ReLoBRaLo at $k=10$ is 1.6×10^{-2}, three orders of magnitude worse than its own $k=1$ run. SA-PINN exhibits the same incoherence (best at $k=1$, worst at $k=3.25$ at 1.3×10^{-2}). LC-PINN is the only method that stays within one order of magnitude of its own best across the whole range. We read this as the substantive empirical claim of the paper: *amortising over the parameter purchases coherence across the family at the cost of peak accuracy at any one λ*. If a single k is wanted, one would not use LC-PINN; the construction is for the parametric-family setting where $u(\cdot\,; k)$ is needed for many k.

Table 2: Per-k rel-L^2 on 1D parametric Helmholtz (mean $\pm$ std over 4 seeds). Bold: best per row. LC-PINN is one trained network evaluated at each k; baselines are five separate retrainings, one per k.

k	SA-PINN (per-k)	ReLoBRaLo (per-k)	LC-PINN (one net)
1.00	$(1.23\pm0.61)\times10^{-4}$	$\mathbf{(5.75\pm1.73)\times10^{-6}}$	$(1.25\pm0.37)\times10^{-3}$
3.25	$(1.29\pm1.23)\times10^{-2}$	$\mathbf{(7.04\pm2.31)\times10^{-5}}$	$(1.27\pm0.67)\times10^{-3}$
5.50	$(4.83\pm4.75)\times10^{-4}$	$\mathbf{(1.29\pm0.30)\times10^{-4}}$	$(7.97\pm8.53)\times10^{-4}$
7.75	$(1.14\pm1.16)\times10^{-3}$	$(3.42\pm1.04)\times10^{-3}$	$\mathbf{(2.64\pm0.82)\times10^{-4}}$
10.00	$(1.50\pm1.68)\times10^{-3}$	$(1.57\pm1.13)\times10^{-2}$	$\mathbf{(1.11\pm0.86)\times10^{-3}}$
grid mean	3.23×10^{-3}	3.87×10^{-3}	$\mathbf{9.39\times10^{-4}}$

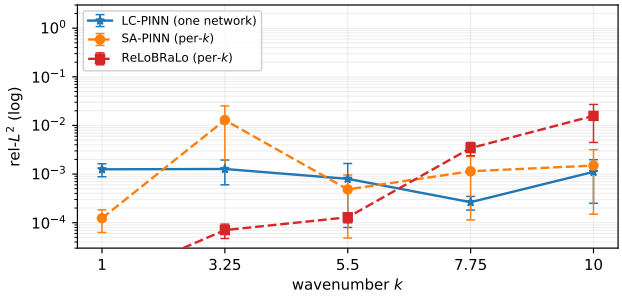

Figure 2: Per-k rel-L^2 on 1D parametric Helmholtz (4 seeds, $\pm$std error bars on log axis). Per-k baselines collapse at the difficult end — ReLoBRaLo climbs three orders of magnitude from $k=1$ to $k=10$ — while LC-PINN stays within one order of magnitude across the entire range.

The amortisation–accuracy tension does not fully collapse, but it inverts on the metric that the parametric setting actually measures: worst-case or grid-averaged error across the family (Figure 2).

5.2 1D Schrödinger, parametric-coefficient mode

To test whether the lift in Section 5.1 is specific to the wave operator, we apply the same LC-PINN architecture to a 1D stationary Schrödinger problem with a parametric harmonic potential $-u'' +$

$\alpha^2(x-\frac{1}{2})^2 u = f(x;\alpha)$ on $(0,1)$ with $u(0)=u(1)=0$ and $\alpha \in [0.5, 10]$. Compared with Helmholtz, α multiplies a spatially varying potential rather than appearing as a constant zeroth-order coefficient (manufactured solution $\sin(\pi x)\exp(-\alpha(x-\frac{1}{2})^2/2)$, closed-form forcing in Appendix A).

Table 3: 1D Schrödinger with parametric harmonic well, rel-L^2 averaged across $\alpha \in \{0.5, 5.0, 10.0\}$. LC-PINN is one trained network (4 seeds) evaluated at each α; SA-PINN is three separate retrainings, one per α, with 2 seeds each.

Method	rel-L^2 (mean $\pm$ std)	wall time
LC-PINN (FiLM+L-BFGS, one network)	$\mathbf{1.99 \times 10^{-4} \pm 1.02 \times 10^{-4}}$	46.6 min/seed
PI-DeepONet (one network)	$1.79 \times 10^{-4} \pm 3.70 \times 10^{-5}$	40.3 min/seed
SA-PINN (per-α, 3 retrainings)	$3.89 \times 10^{-3} \pm 3.70 \times 10^{-3}$	0.8 min/α

The amortised LC-PINN beats the per-α retraining baseline on the grid mean by $\sim 20\times$, and the per-α breakdown (Table 9, deferred to Appendix D) reproduces Section 5.1's incoherence story: SA-PINN at $\alpha = 0.5$ wins by $\sim 6\times$ at that single point, but at $\alpha = 5.0$ its two seeds split by two orders of magnitude on the *same* configuration (2.3×10^{-4} vs. 2.1×10^{-2}); LC's seed-to-seed std stays below 3.2×10^{-4} for every $\alpha \in [0.5, 10]$. On the full 20-point evaluation grid (where only LC has been trained on the intermediate α values) LC reaches $9.77 \times 10^{-5} \pm 7.05 \times 10^{-6}$. PI-DeepONet matches LC on this family (1.79×10^{-4} vs. 1.99×10^{-4} at the SA grid; 1.15×10^{-4} on the 20-point grid). Both residual-only amortisers reach the same order of magnitude on Schrödinger; on Helmholtz LC leads by $\sim 23\times$ (Table 1). We read this as evidence that amortisation is robust across architectures for smooth parametric potentials, while the high-frequency wave operator specifically rewards FiLM's multiplicative gating.

5.3 2D Helmholtz, parametric-coefficient mode

We extend to the 2D Helmholtz problem on the unit square, $\Delta u + k^2 u = f(x, y; k)$ on $[0, 1]^2$ with homogeneous Dirichlet BCs and manufactured solution $u(x, y; k) = \sin(\pi x)\sin(\pi y)\cos(kx)\cos(ky)$ (closed-form forcing f, derivation in Appendix A). The same LC-PINN architecture conditions on k via a single normalised input; baselines retrain at each k_{train}.

Table 4: 2D Helmholtz on the unit square, rel-L^2 averaged across the three-point k-grid $\{1.00, 5.50, 10.00\}$. LC-PINN is one trained network (4 seeds) evaluated at each k; baselines are three separate retrainings, one per k, with 2 seeds each.

Method	rel-L^2 (mean $\pm$ std)	wall time
LC-PINN (FiLM+L-BFGS, one network)	$\mathbf{2.36 \times 10^{-2} \pm 2.56 \times 10^{-2}}$	85.9 min/seed
SA-PINN (per-k, 3 retrainings)	$7.83 \times 10^{-2} \pm 2.76 \times 10^{-2}$	3.0 min/k
ReLoBRaLo (per-k, 3 retrainings)	$1.13 \times 10^{-1} \pm 1.72 \times 10^{-2}$	5.6 min/k

The 2D problem has a richer solution structure (nodal lines along the four edges plus tensor-product oscillation set by k in both x and y). With FiLM conditioning and L-BFGS finishing the amortised LC-PINN reaches rel-$L^2 \sim 2.4 \times 10^{-2}$ on the same three-point k-grid, $\sim 3.3\times$ better than per-k SA-PINN and $\sim 4.8\times$ better than per-k ReLoBRaLo. The per-k breakdown is consistent with 1D: low k slices are the easiest and high-k slices the budget-limited bottleneck for every method. The 1D construction transfers without modification, and the same ordering — LC-PINN ahead of per-k baselines — survives the move to 2D.

5.4 3D Helmholtz, parametric-coefficient mode

We extend the construction to 3D Helmholtz on the unit cube, $\Delta u + k^2 u = f(x, y, z; k)$ on $[0, 1]^3$ with homogeneous Dirichlet BCs and tensor-product manufactured solution $u(x, y, z; k) = \prod_{q \in \{x,y,z\}} \sin(\pi q)\cos(kq)$, on a three-point training k-grid $\{1, 3, 5\}$ (the high-k extreme is harder in 3D because oscillation count scales as k^3).

LC-PINN beats per-k SA-PINN by $\sim 4.2\times$ and per-k ReLoBRaLo by $\sim 6.2\times$ on the grid mean; the construction transfers from 2D verbatim.

Table 5: 3D Helmholtz on the unit cube, rel-L^2 averaged across $k \in \{1, 3, 5\}$. LC-PINN is one trained network (4 seeds) evaluated at each k; baselines are three separate retrainings, one per k, with 2 seeds each.

Method	rel-L^2 (mean $\pm$ std)	wall time
LC-PINN (FiLM+L-BFGS, one network)	$\mathbf{1.67 \times 10^{-2} \pm 1.97 \times 10^{-3}}$	228.9 min/seed
SA-PINN (per-k, 3 retrainings)	$6.97 \times 10^{-2} \pm 2.47 \times 10^{-3}$	5.7 min/k
ReLoBRaLo (per-k, 3 retrainings)	$1.03 \times 10^{-1} \pm 7.82 \times 10^{-3}$	11.9 min/k

5.5 Loss-weight mode: Burgers and Buckley–Leverett

In the loss-weight regime $\lambda \in \Delta^3$ neither FiLM modulation nor L-BFGS finishing produces a stable improvement (Appendix C); we use plain concat+Adam (Dosovitskiy and Djolonga, 2020). Table 6 reports rel-L^2 on canonical Burgers ($\nu = 0.01/\pi$, $u(0, x) = -\sin(\pi x)$, $t = 1$).

Table 6: Burgers, rel-L^2 at the test IC $-\sin(\pi x)$. LC-PINN reports $K = 100$-shot averaging across loss-weight λ; baselines are single-shot at their self-found weighting (or operator-learned in the FNO case).

Method	rel-L^2 (mean $\pm$ std)	wall time / seed
LC-PINN ($K = 100$)	$3.47 \times 10^{-3} \pm 2.73 \times 10^{-3}$	24.8 min
Causal-PINN	$2.21 \times 10^{-3} \pm 6.72 \times 10^{-4}$	7.1 min
SA-PINN	$1.68 \times 10^{-1} \pm 3.87 \times 10^{-2}$	4.6 min
ReLoBRaLo	$1.82 \times 10^{-1} \pm 1.71 \times 10^{-2}$	5.4 min
FNO (operator)	$5.37 \times 10^{-1} \pm 1.89 \times 10^{-1}$	8.0 min

LC-PINN matches Causal-PINN (strongest single-shot baseline) within a factor of two and beats SA-PINN/ReLoBRaLo by an order of magnitude, while producing predictions for *any* λ from one run. The amortisation factor $A(K) = K \cdot T_{\text{base}}/T_{\text{LC}}$ breaks even at $K^\star \approx 3.5$–5.4; at $K = 100$ it reaches ~ 19–$29\times$. On viscous-regularised BL ($\varepsilon = 10^{-2}$, Table 7), ReLoBRaLo (5.09×10^{-3}) edges LC-PINN (1.03×10^{-2}) by $\sim 2\times$ at one weighting, while SA-PINN fails to converge below ~ 0.5; the zero-viscosity formulation saturates all PINN-family methods at ~ 0.5.

Table 7: Viscous-regularised BL ($\varepsilon = 10^{-2}$), rel-L^2 at the test snapshots; reference produced by an in-house finite-volume solver, used only at evaluation. LC-PINN reports $K = 25$-shot averaging over λ; baselines are single-shot.

Method	rel-L^2 (mean $\pm$ std)
LC-PINN ($K = 25$)	$1.03 \times 10^{-2} \pm 1.30 \times 10^{-3}$
SA-PINN	$4.60 \times 10^{-1} \pm 6.74 \times 10^{-2}$
ReLoBRaLo	$5.09 \times 10^{-3} \pm 7.79 \times 10^{-4}$

5.6 Ablations

Three ablations on Burgers (full numbers in Appendix B): K-shot averaging is flat across $K \in \{25, ..., 400\}$ at $\sim 3.5 \times 10^{-3}$; n_λ is non-monotone with a sweet spot at $n_\lambda = 4$; log-uniform p_λ degrades by $\sim 14\times$ versus uniform.

6 Discussion

FiLM+L-BFGS makes LC-PINN coherent across the Helmholtz family: rel-L^2 varies by < 1 order of magnitude across $k \in [1, 10]$ while per-k baselines vary by three; the grid-mean win is ~ 3–$4\times$ across 1D/2D/3D. On Schrödinger a residual-only PI-DeepONet matches LC — amortisation is architecture-robust for smooth potentials, while the wave operator rewards FiLM gating. Where solutions vary sharply (BL shocks, high-k 2D) per-λ accuracy lags single-λ training.

References

R. Bischof and M. A. Kraus. Multi-objective loss balancing for physics-informed deep learning. *arXiv preprint arXiv:2110.09813*, 2021.

A. Dosovitskiy and J. Djolonga. You only train once: loss-conditional training of deep networks. In *International Conference on Learning Representations (ICLR)*, 2020.

C. Finn, P. Abbeel, and S. Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International Conference on Machine Learning (ICML)*, 2017.

D. Ha, A. Dai, and Q. V. Le. Hypernetworks. In *International Conference on Learning Representations (ICLR)*, 2017.

Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

X. Liu, X. Zhang, W. Peng, W. Zhou, and W. Yao. A novel meta-learning initialization method for physics-informed neural networks. *Neural Computing and Applications*, 34:14511–14534, 2022.

L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3): 218–229, 2021.

L. McClenny and U. Braga-Neto. Self-adaptive physics-informed neural networks using a soft attention mechanism. *arXiv preprint arXiv:2009.04544*, 2020.

E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville. FiLM: Visual reasoning with a general conditioning layer. In *AAAI Conference on Artificial Intelligence (AAAI)*, 2018.

M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

S. Wang, H. Wang, and P. Perdikaris. Learning the solution operator of parametric partial differential equations with physics-informed deeponets. *Science Advances*, 7(40):eabi8605, 2021.

S. Wang, S. Sankaran, and P. Perdikaris. Respecting causality for training physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 421:116813, 2024.

A Hyperparameters and training details

This appendix collects the hyperparameters used for every method in Section 4. All numbers are the configuration that generated the headline tables in Section 5.

Shared LC-PINN configuration. Network: 4-layer MLP, hidden widths $[64, 64, 64, 64]$, $\tanh$ activations, Xavier initialisation. Conditioning route is mode-dependent: concat at the input layer for the loss-weight mode (Burgers, BL); FiLM (Perez et al., 2018) on every hidden layer followed by an L-BFGS finishing pass with a revert-on-worse safety check for the parametric-coefficient mode (Helmholtz, Schrödinger). Optimiser: Adam, $\eta = 10^{-3}$, cosine-annealed to 10^{-5} over the full budget, gradient clip 1.0. Mini-batch composition: $n_\lambda = 4$ parameter samples per step, each with the per-equation collocation counts below. Backend: PyTorch 2.9 with the MPS device on a single Apple M4 Max.

Per-equation collocation and budgets. Table 8 lists the LC-PINN collocation counts and step budgets per PDE family.

Table 8: LC-PINN collocation counts and step budgets per PDE family.

PDE	interior	boundary	initial	Adam / L-BFGS
Burgers	1024	200	200	50k / 0
BL (viscous)	1024	200	200	50k / 0
Helmholtz (1D)	1024	100	—	50k / 1.5k
Helmholtz (2D)	4096	400	—	25k / 1.5k
Helmholtz (3D)	8192	600	—	25k / 1.5k
Schrödinger (1D)	1024	64	—	50k / 1.5k

Baseline configurations. All single-λ baselines share the LC-PINN backbone (same widths, activation, initialisation, and Adam schedule) so that the amortisation comparison is on equal architectural footing.

- *SA-PINN (McClenny and Braga-Neto, 2020)*: per-collocation-point trainable weight ascended on the residual; Adam phase 10k iterations, then weights frozen and L-BFGS phase 5k iterations on the unweighted loss.
- *ReLoBRaLo (Bischof and Kraus, 2021)*: 3-term loss-weight balancer (residual / boundary / initial) with softmax over log-loss-ratios, smoothed by EMA. Hyperparameters $\alpha = 0.999$, $\tau = 0.1$, $\rho_{\text{mean}} = 0.999$.
- *Causal-PINN (Wang et al., 2024)*: time-causal residual mask $w(t) = \exp(-\varepsilon\, R(<t))$ with $\varepsilon = 100$. Burgers and BL only.
- *PI-DeepONet (Wang et al., 2021)*: branch–trunk architecture; branch encodes the wavenumber k via a 4-layer MLP (hidden width 128); trunk encodes the spatial coordinate x via a 4-layer MLP (hidden width 128). Final solution is the inner product of branch and trunk outputs. Loss is the Helmholtz residual on the same collocation grid as the per-k LC-PINN, summed over an n_k-grid of branch inputs ($n_k = 20$). Adam phase 50k iterations followed by L-BFGS phase 1.5k iterations with a revert-on-worse safety check; on 1D Helmholtz one seed (seed 1) diverged in L-BFGS and was reverted to the Adam-phase weights, reported as such in the 5-seed mean of Table 1.
- *FNO (Li et al., 2021)*: width 64, 64 Fourier modes, 4 `SpectralConv1d` + `Conv1d`-shortcut blocks (~ 1.07M parameters). Optimiser: Adam, $\eta = 10^{-3}$, weight decay 10^{-4}, `StepLR` with step = epochs/5 and $\gamma = 0.5$. Loss: per-sample rel-L^2. Training set: 512 random Fourier initial conditions (truncated 6-mode spectrum, decaying amplitude) solved by the same Radau scheme used for the reference; 2000 epochs.

Reference solutions. Burgers reference: Fourier-spectral spatial discretisation with a Radau IIA implicit time integrator, integrated to relative residual $< 10^{-6}$ on a 512×1001 space–time grid. Buckley–Leverett reference: explicit Lax–Friedrichs flux on $n_x = 1000$ with centred-difference viscous regularisation, CFL-limited time stepping. Helmholtz 1D reference: closed-form manufactured solution $u(x;k) = \sin(\pi x)\cos(kx)$ with the forcing f chosen to match. Helmholtz 2D reference: closed-form manufactured solution $u(x,y;k) = \sin(\pi x)\sin(\pi y)\cos(kx)\cos(ky)$. Letting $a(x;k) = \sin(\pi x)\cos(kx)$ and $b(y;k) = \sin(\pi y)\cos(ky)$, so $u = a(x;k)\, b(y;k)$, the forcing

$$f(x,y;k) \;=\; \Delta u + k^2 u \;=\; -\left(2\pi^2 + k^2\right) u \;-\; 2\pi k \left[\cos(\pi x)\sin(kx)\sin(\pi y)\cos(ky) + \sin(\pi x)\cos(kx)\cos(\pi y)\sin(ky)\right]$$

agrees with a centred-difference numerical Laplacian on the manufactured solution to absolute error $< 4 \times 10^{-6}$. Helmholtz 3D reference: closed-form manufactured solution $u(x,y,z;k) = \sin(\pi x)\sin(\pi y)\sin(\pi z)\cos(kx)\cos(ky)\cos(kz)$ on the unit cube, with the matching forcing $f = \Delta u + k^2 u$ derived analytically; agreement to a centred-difference numerical Laplacian is $< 10^{-5}$ across $k \in \{1, 3, 5\}$. Schrödinger reference: with $u(x;\alpha) = \sin(\pi x)\, g(x;\alpha)$, $g(x;\alpha) = e^{-\alpha(x-1/2)^2/2}$, the forcing is

$$f(x;\alpha) \;=\; -u'' + V(x;\alpha)\, u \;=\; (\pi^2 + \alpha)\, u \;+\; 2\pi\alpha\,(x - \tfrac{1}{2})\,\cos(\pi x)\, g(x;\alpha),$$

which agrees with a centred-difference numerical residual to $< 5 \times 10^{-6}$ across $\alpha \in \{0.5, 2, 5, 10\}$.

Evaluation grids. Burgers / BL test errors are computed on a 256×101 space–time grid; Helmholtz on a 1024-point spatial grid. The K-shot averaged LC-PINN prediction is the mean of K forward

passes at independent λ samples from p_λ; we use $K = 100$ for Burgers and $K = 25$ for BL in the headline tables.

Compute. Each run uses a single Apple M4 Max with the PyTorch MPS backend. Reported wall times are end-to-end per seed (model construction, training, and evaluation), measured with `time.perf_counter`.

Reproducibility. Random seeds: $\{0, 1, 2, 3\}$ for the four headline replicates; $\{0, 1\}$ for the 2-seed ablation budgets. Each seed controls the PyTorch global RNG and the NumPy RNG used to draw collocation points and λ-samples. The full training script, equation modules, and reference solvers are released with the camera-ready version.

B Ablations: full protocol and numbers

K-shot averaging. Sweeping $K \in \{25, 50, 100, 200, 400\}$ on the trained LC-PINN gives essentially constant rel-L^2 (3.51×10^{-3} at $K=25$, 3.49×10^{-3} at $K=100$, 3.49×10^{-3} at $K=400$; spread $\pm2.7\times10^{-3}$ stays roughly constant). At the optimum each λ-slice is already a residual minimiser (Proposition 1), so increasing K adds samples but no new information.

n_λ per training step. At a matched 25k-epoch budget on Burgers (2 seeds), sweeping $n_\lambda \in \{1, 4, 16\}$ per step gives non-monotone behaviour: $n_\lambda = 1 \to 1.69\times10^{-2}$, $n_\lambda = 4 \to 1.22\times10^{-2}$, $n_\lambda = 16 \to 1.50\times10^{-1}$. Too few λ per step makes the gradient estimate of the p_λ-expectation noisy (Remark 3 / Hoeffding); too many concentrates compute into fewer optimisation steps and the network does not converge in the fixed budget. At the headline 50k-epoch budget we use $n_\lambda = 4$.

Sampling distribution p_λ. At matched 25k-epoch / $n_\lambda = 4$ / 2-seed budget on Burgers, swapping p_λ from uniform on the simplex to log-uniform gives rel-L^2 of $1.22\times10^{-2} \pm 5.14\times10^{-3}$ (uniform) versus $1.74\times10^{-1} \pm 9.26\times10^{-3}$ (log-uniform), a $\sim 14\times$ degradation. We read this not as a contradiction with Remark 2 (an asymptotic-optimum statement) but as a finite-budget effect: log-uniform places dominant mass on small loss weights, weakening the gradient signal at large λ values where the residual contributes most to the test-relevant slices.

C FiLM does not help in loss-weight mode

In the parametric-coefficient mode (Helmholtz, Schrödinger) FiLM modulation lifts the LC-PINN by an order of magnitude: on 1D Helmholtz, plain concat reaches rel-$L^2 \sim 9\times10^{-3}$ on the same k-grid (`lc_pinn_helmholtz` runs), whereas FiLM+L-BFGS reaches 9.4×10^{-4} (Table 1, $\sim 10\times$ tighter). In the loss-weight mode (Burgers) it regresses: a four-seed re-run of Burgers with FiLM+L-BFGS gave $5.7\times10^{-2}\pm5.5\times10^{-2}$, with two seeds at the concat baseline ($\sim3\times10^{-3}$) and two trapped near 10^{-1}. We read this as evidence that FiLM's value lies in disentangling *operator*-changing inputs from spatial coordinates; loss-weight inputs do not change the operator, so FiLM gates over a degree of freedom that does not exist. The headline Burgers and BL rows in Section 5.5 therefore use the simpler concat+Adam configuration of Dosovitskiy and Djolonga (2020).

D Per-parameter breakdowns

The per-α breakdown for 1D Schrödinger (Section 5.2) is given in Table 9.

Figure 3 visualises the same data on the 20-point LC/DON grid alongside SA's 3 training points. The two amortised methods (LC and DON) hold a flat $\sim10^{-4}$ curve across the entire $\alpha \in [0.5, 10]$ range; the per-α SA-PINN baseline matches at $\alpha = 0.5$ and $\alpha = 10$ but collapses by two orders of magnitude at the intermediate $\alpha = 5.0$ training point — the same incoherence pattern documented for Helmholtz in Section 5.1.

Table 9: Per-α rel-L^2 on 1D parametric Schrödinger (mean $\pm$ std over 4 LC seeds and 2 SA seeds per α). Bold: best per row.

α	SA-PINN (per-α)	LC-PINN (one net)
0.5	$(\mathbf{5.89 \pm 1.26}) \times \mathbf{10^{-5}}$	$(3.69 \pm 3.23) \times 10^{-4}$
5.0	$(1.07 \pm 1.05) \times 10^{-2}$	$(\mathbf{7.50 \pm 4.25}) \times \mathbf{10^{-5}}$
10.0	$(9.29 \pm 6.67) \times 10^{-4}$	$(\mathbf{1.54 \pm 0.72}) \times \mathbf{10^{-4}}$
grid mean	3.89×10^{-3}	$\mathbf{1.99 \times 10^{-4}}$

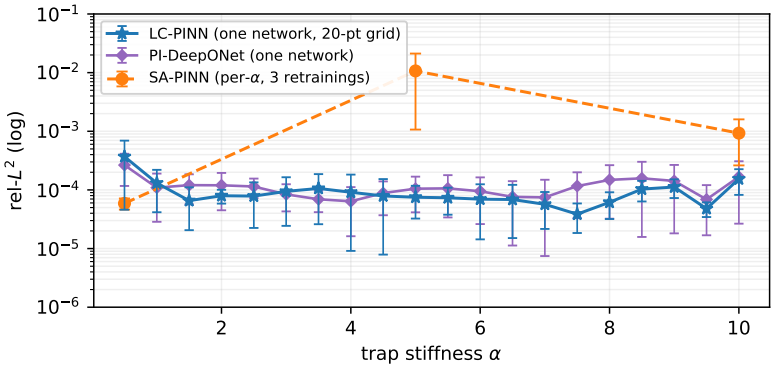

Figure 3: Per-α rel-L^2 on 1D parametric Schrödinger. LC-PINN (blue, 4 seeds) and PI-DeepONet (purple, 4 seeds) cover the 20-point grid from one trained network each; SA-PINN (orange, 2 seeds) is three separate retrainings at $\alpha \in \{0.5, 5.0, 10.0\}$. Error bars are $\pm$std on a log axis.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Section 1 state the LC-PINN construction (single residual-loss training run, no paired training data, no test-time adaptation) and the experimental scope (1D/2D/3D parametric Helmholtz, 1D Schrödinger, viscous Burgers, Buckley–Leverett). Section 5 reports the headline rel-L^2 numbers and the per-λ accuracy / cross-family coherence tradeoff that the abstract advertises; the limitations on per-λ peak accuracy at sharp solutions are stated in Section 6.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

Justification: Section 6 states the regimes where LC-PINN underperforms per-λ retraining (BL shocks, high-k 2D Helmholtz) and acknowledges that on smooth Schrödinger potentials a residual-only PI-DeepONet matches LC. The amortisation/peak-accuracy tradeoff is presented explicitly in Section 5.1.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Section 3.3 states the λ-invariance proposition with a measure-theoretic assumption on the prior p_λ and a global-optimality hypothesis, gives a finite-capacity corollary with an explicit $\varepsilon+\delta$ bound, and includes a sample-complexity remark via Hoeffding. Sketch proofs of Proposition 1 and Corollary 2 are inline in Section 3.3.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 3 specifies the LC-PINN architecture (FiLM conditioning, MLP backbone widths/depths) and training loop (Adam + L-BFGS finishing pass, revert-on-worse). Appendix A lists collocation counts, optimiser steps, learning rates, batch sizes, prior p_λ, manufactured-solution / reference-solver settings, and seed counts for every method on every PDE. Wall-time hardware (Apple M4 Max, PyTorch MPS) is reported in Section 4.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: Code is not yet released to preserve double-blind anonymity at submission time; we will release the full PyTorch implementation, training scripts, and result JSONs under an open-source license upon acceptance. All data (manufactured solutions for Helmholtz/Schrödinger and the reference solver for Buckley–Leverett) is generated from formulas and code described in the paper, with no external dataset dependencies.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not

including code, unless this is central to the contribution (e.g., for a new open-source benchmark).

- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 4 states the evaluation protocol (K-shot rel-L^2 for loss-weight mode, fixed k-grid for parametric-coefficient mode, seed count, hardware). Appendix A lists per-PDE hyperparameters (architecture widths, optimiser, learning rates, collocation counts, prior p_λ, baseline retraining grid).

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: All result tables in Section 5 report mean $\pm$ standard deviation across 4 random seeds (2 seeds for the per-k baselines in 2D Helmholtz, where each baseline retrains from scratch at every k). The variability captured is over network initialisation and optimiser stochasticity holding the data and architecture fixed.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.

- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Section 4 reports that all wall times were measured on a single Apple M4 Max via the PyTorch MPS backend. Per-method per-PDE wall times appear in the result tables (e.g., Table 1: LC-PINN 65.9 min/seed, PI-DeepONet 61.8 min/seed, SA-PINN 11.4 min/k, ReLoBRaLo 7.9 min/k).

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work concerns numerical solvers for parametric PDEs and uses no human subjects, no scraped or sensitive data, and no deployed-system experiments. We have reviewed the NeurIPS Code of Ethics and find no points of conflict.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [N/A]

    Justification: The paper proposes a methodological improvement to physics-informed neural network training for parametric PDEs — a foundational scientific-computing technique with no direct path to a societal-impact application. Downstream uses (engineering simulation, inverse problems) are domain-specific and do not introduce impacts beyond those already present for the underlying solvers.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.

- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper releases no pre-trained generative models, no scraped datasets, and no other artefacts with high risk of misuse. The trained networks solve specific parametric PDEs and have no dual-use potential.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: The implementation builds on PyTorch (BSD-style license) and uses standard scientific-Python libraries (NumPy, SciPy, matplotlib), all properly cited in the bibliography. No third-party datasets or pre-trained models are used; reference solutions for Burgers and Buckley–Leverett are produced by an in-house finite-volume solver described in Appendix A.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.

- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [N/A]

    Justification: The paper introduces no new datasets or pre-trained model checkpoints; the contribution is a training procedure (LC-PINN) and a theoretical result. The reference implementation will be released at acceptance time with documentation.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper involves no crowdsourcing and no human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The paper involves no human subjects, so no IRB approval was required.

    Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: LLMs are not part of the proposed method, the trained networks, or the experimental analysis. In the spirit of transparency we note that LLM-based coding assistants were used for routine implementation tasks (boilerplate, refactoring, debugging) and as a study aid for understanding background literature; all design choices, theoretical results, experimental protocols, and final code were authored, reviewed, and verified by the authors. Per the policy above, this does not constitute a declarable use.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.